

Week 4 Preview — Functions, Decomposition and Abstraction

What we'll learn

When you use a smartphone, you don't need to know how its circuits work — you touch the screen and get the result you want. The lecture notes call this a "black box." This week we learn to package code into such black boxes, called **functions**: splitting a big problem into self-contained pieces (decomposition) while hiding the tedious details (abstraction). We also get a first taste of the idea that a function is itself an object, just like a number.

Terms to know

Term	Meaning
Abstraction	Hiding how something works inside; you only need the relationship between input and output
Decomposition	Splitting code into self-contained, reusable pieces (modules)
Function	A reusable piece of code that captures the steps of a computation
Parameter	An input to a function (a "formal parameter" in the definition)
Return	Giving a value back to the caller, using the keyword <code>return</code>
Docstring	The <code>"""..."""</code> specification of a function — a "contract" between coder and user

Try this code

This is the `is_even` example from Lecture 7. Type it in and run it.

```
def is_even(i):  
    """  
    Input: i, a positive int  
    Returns True if i is even, otherwise False  
    """  
    return i % 2 == 0  
  
print(is_even(3))  
print(is_even(8))  
  
my_func = is_even      # a function bound to a new name (taste of Lecture 8)  
print(my_func(4))
```

Predict before you run

1. What will each of the three `print` lines display? Write your guesses first.
2. Why does `my_func(4)` work exactly like `is_even(4)` after the assignment?
3. If the file contained only the `def` block and no call like `is_even(3)`, what would appear on the screen?

Questions to think about

1. What is the difference between *defining* a function (`def`) and *calling* (invoking) it? Explain in your own words: "a function's code only runs when you call it."
2. Smartphone component makers build a working phone without knowing each other's internal designs. What are two benefits of splitting a program into functions in the same way?

Source

- MIT OpenCourseWare, 6.100L *Introduction to CS and Programming Using Python* (Fall 2022), Lecture 7 "Decomposition, Abstraction, Functions" and Lecture 8 "Functions as Objects" lecture notes.
- License: CC BY-NC-SA (<https://ocw.mit.edu/terms>)